GESTIONE MAGAZZINO MOTO

Documentazione Tecnica Completa

Frontend + Backend — Versione 1.1

Frontend	HTML · CSS · JavaScript
Backend	Node.js · Express · SQLite
Real-time	Socket.IO
Database	SQLite (magazzino.db)

1. Panoramica dell'Applicazione

Gestione Magazzino Moto è un sistema web completo per la gestione del magazzino ricambi di un'officina o rivendita moto. Permette di gestire marche, prodotti, movimenti di carico e scarico, visualizzare le giacenze con logica FIFO e generare stampe professionali.

Il sistema è diviso in due parti che lavorano insieme:

- Frontend: le pagine HTML/CSS/JS con cui interagisce l'utente
- Backend: il server Node.js che gestisce i dati e le API

Come accedere

Campo	Valore
URL	http://IP-SERVER:3000
Utente default	Admin
Password default	Admin123!
Health check	http://IP-SERVER:3000/api/health

Struttura delle cartelle

Cartella / File	Contenuto
frontend/	Client: pagine HTML, CSS e JavaScript, organizzato per funzionalità
frontend/index.html · home.html	Pagina di login e applicazione principale (sidebar + sezioni)
frontend/assets/	Risorse statiche: img/ (loghi) e company-info.json (dati aziendali)
frontend/css/base/app.css	Stili dell'applicazione (home.html)
frontend/css/auth/css/auth/ login.css	Stili della pagina di login
frontend/css/features/	Stili delle nuove funzionalità (dashboard.css)
frontend/css/shared/	Stili dei componenti condivisi (bulk-selection.css)
frontend/js/core/	Base dell'app: config, utils, realtime, company-loader, navigation
frontend/js/shared/	Componenti riusati da più funzionalità: ui/ (notifiche, modali, helper form), search.js, bulk-selection.js
frontend/js/features/	Una cartella per funzionalità: auth, dashboard, marche, prodotti, movimenti, riordino, riepilogo, utenti, stampe
Backend/server.js	Punto di ingresso del server (HTTP + Socket.IO)
Backend/src/app.js	Configurazione Express (middleware, static, route)
Backend/src/config/	Connessione SQLite e schema delle tabelle (schema.js)
Backend/src/routes/	Definizione degli endpoint API (incluse dashboardRoutes e le route bulk-delete)

Cartella / File	Contenuto
Backend/src/controllers/	Leggono la richiesta HTTP e chiamano il service
Backend/src/services/	Logica di business (FIFO, movimenti, dashboard, eliminazione multipla)
Backend/src/repositories/	Query SQL (una per entità + dashboardRepository)
Backend/src/realtime/socket.js	Socket.IO: emit, batch() e notifiche movimenti
Backend/src/utils/	decimal.js, network.js e bulkDelete.js (eliminazione multipla)
Backend/src/scripts/seed.js	Script opzionale per dati di esempio
Backend/db/magazzino.db	Database SQLite (creato al primo avvio)
Documenti/	Documentazione tecnica e manuale utente (Word e PDF)

2. Frontend

Pagine

File	CSS	Funzione
index.html	css/auth/login.css	Login con form username/password e dati aziendali in calce
home.html	css/base/app.css	Dashboard con sidebar e 8 sezioni navigabili

Sezioni della Dashboard

La dashboard è divisa in sezioni navigabili dalla sidebar. Solo una sezione è visibile alla volta, attivata tramite la classe CSS active.

Sezione	data-section	Funzione
🧭 Dashboard	dashboard	Pagina iniziale: cosa fare adesso, numeri chiave, grafico e ultimi movimenti
Marche	marche	Crea e gestisce le marche (es. Honda, Yamaha). Mostra il numero di prodotti collegati per marca.
Prodotti	prodotti	Anagrafica prodotti con codice, marca, descrizione e giacenza attuale.
Movimenti	movimenti	Registra carichi (entrata merce) e scarichi (uscita merce). Filtrabili per testo e per range di date con freccia visiva tra i due campi data.
Riordino	riordino	Mostra tutti i prodotti con giacenza zero da riordinare. Cliccando Riordina si apre direttamente il modal movimento precompilato.
Riepilogo	riepilogo	Giacenze attuali per ogni prodotto con dettaglio lotti FIFO e valore economico totale del magazzino.
Storico	storico	Consulta le giacenze come erano in una data passata, selezionabile da calendario.
Utenti	utenti	Gestisce gli account: crea, modifica username/password, elimina. Se si elimina o modifica il proprio account viene eseguito il logout automatico.

Script JavaScript — Ordine di caricamento (home.html)

L'ordine è obbligatorio: ogni file dipende da quelli precedenti.

#	File	Cosa fa
1	socket.io (CDN)	Libreria Socket.IO per il real-time
2	core/realtime.js	Connessione Socket.IO, eventi real-time e aggiornamento della Dashboard
3	core/config.js	API_URL = "api" e variabili di stato globali
4	core/utils.js	formatCurrency, formatQuantity, escapeHtml, limitToTwoDecimals, forceLogout

#	File	Cosa fa
5	shared/ui/notifications.js	Toast (showNotification), showConfirmModal, showAlertModal
6	shared/ui/formHelpers.js	Helper dei form (toggle password, campo prezzo)
7	shared/ui/modals.js	Apertura/chiusura dei modali di marche, prodotti, movimenti, utenti
8	shared/search.js	Filtri per ogni sezione con persistenza su localStorage
9	shared/bulk-selection.js	Selezione multipla ed eliminazione in blocco (NUOVO)
10	core/company-loader.js	Carica assets/company-info.json e popola sidebar, login, stampe
11	features/dashboard/dashboard.js	Dashboard: cose da fare, KPI, grafico, ultimi movimenti (NUOVO)
12	features/marche/marche.js	CRUD marche + selezione multipla
13	features/prodotti/prodotti.js	CRUD prodotti + selezione multipla
14	features/movimenti/render.js · search.js · data.js	Tabella movimenti, filtri, caricamento dati + selezione multipla
15	features/riordino/riordino.js	Sezione Riordino (prodotti esauriti)
16	features/riepilogo/riepilogo.js	Giacenze FIFO, storico a data, valore magazzino
17	features/utenti/utenti.js	CRUD utenti + selezione multipla
18	features/stampe/pdf-scarico.js · pdf-fattura.js	Import scarichi da PDF e carico da fattura PDF con AI
19	core/navigation.js	Bootstrap: navigazione sezioni (Dashboard di default), hamburger, logout
20	features/stampe/stampe.js	Stampa riepilogo e storico con intestazione aziendale

Sistema di Ricerca e Filtri (search.js)

Ogni sezione ha un campo di ricerca il cui valore viene salvato in localStorage e ripristinato automaticamente quando si torna sulla sezione.

Sezione	Chiave localStorage
Marche	search_marche
Prodotti	search_prodotti
Movimenti (testo)	search_movimenti
Movimenti — Data inizio	search_movimenti_start
Movimenti — Data fine	search_movimenti_end
Riepilogo	search_riepilogo
Storico	search_storico
Utenti	search_utenti

Riordino	search_riordino

Dati Aziendali (assets/company-info.json)

Modificando questo file si aggiornano automaticamente: l'indirizzo e P.IVA nella pagina login, i dati nella sidebar, e l'intestazione di tutte le stampe.

Campo	Descrizione
company.address	Via e numero civico
company.city / cap / province	Città, CAP e sigla provincia
company.piva	Partita IVA
company.email / phone	Email e telefono per le stampe
company.name / logo	Nome azienda e percorso del logo
settings.currency / currencySymbol	Valuta e simbolo (EUR / €)
settings.decimalSeparator	Separatore decimale (, in Italia)

3. Backend

Struttura del server (server.js)

Il server è scritto in Node.js con Express. Gestisce sia le API REST che la distribuzione del frontend e la comunicazione real-time con Socket.IO.

- Porta: 3000 (configurabile con variabile d'ambiente PORT)
- Avvio: node server.js oppure pm2 start server.js
- Il server serve anche i file statici del frontend dalla cartella ../frontend
- Ogni richiesta API viene loggata in console con timestamp, metodo e IP

API Routes

Prefisso	File route	Gestisce
/api/auth	routes/authRoutes.js	Login utenti con verifica bcrypt
/api/marche	routes/marcheRoutes.js	CRUD marche
/api/prodotti	routes/prodottiRoutes.js	CRUD prodotti con giacenza
/api/dati	routes/datiRoutes.js	CRUD movimenti (carico/scarico) e bulk-scarico da PDF
/api/magazzino	routes/ magazzinoRoutes.js	Riepilogo FIFO, storico a data, valore totale
/api/utenti	routes/utentiRoutes.js	CRUD utenti con hashing bcrypt
/api/health	server.js	Health check: stato server, uptime, IP pubblico, connessioni socket

Endpoint API principali

Metodo	Endpoint	Funzione
POST	/api/auth/login	Login: verifica username e password
GET	/api/marche	Lista tutte le marche con conteggio prodotti collegati
POST	/api/marche	Crea nuova marca
PUT	/api/marche/:id	Modifica nome marca
DELETE	/api/marche/:id	Elimina marca (solo se senza prodotti)
GET	/api/prodotti	Lista prodotti con giacenza, marca e descrizione
POST	/api/prodotti	Crea nuovo prodotto
PUT	/api/prodotti/:id	Modifica prodotto
DELETE	/api/prodotti/:id	Elimina prodotto (solo se giacenza = 0 e nessun movimento)
GET	/api/dati	Lista tutti i movimenti
POST	/api/dati	Crea movimento carico o scarico

DELETE	/api/dati/:id	Elimina movimento
POST	/api/dati/bulk-scarico	Scarico multiplo da import PDF
GET	/api/magazzino/riepilogo	Giacenze attuali con lotti FIFO e valore per prodotto
GET	/api/magazzino/storico-giacenza/:date	Giacenze come erano alla data specificata
GET	/api/utenti	Lista utenti (senza password)
POST	/api/utenti	Crea utente (password hashata con bcrypt)
PUT	/api/utenti/:id	Modifica username o password
DELETE	/api/utenti/:id	Elimina utente (non si può eliminare l'ultimo)

4. Database (SQLite)

Il database è un file SQLite salvato in backend/db/magazzino.db. Viene creato automaticamente al primo avvio se non esiste. Viene anche creato automaticamente un utente Admin con password Admin123!

Tabelle

Tabella	Colonne principali	Funzione
marche	id, nome, data_creazione	Categorie/marche dei prodotti
prodotti	id, nome, marca_id, descrizione, data_creazione	Anagrafica prodotti collegati a una marca
dati	id, prodotto_id, tipo, quantita, prezzo, data_movimento, fattura_doc, fornitore_cliente_id	Storico completo di tutti i movimenti (carico e scarico)
lotti	id, prodotto_id, quantita_iniziale, quantita_rimanente, prezzo, data_carico, fattura_doc, fornitore	Lotti fisici per il calcolo FIFO della giacenza
users	id, username, password, createdat	Account utenti con password hashata bcrypt

Logica FIFO

Ogni carico crea un nuovo lotto nella tabella lotti. Ogni scarico sottrae la quantità partendo dal lotto più vecchio (First In First Out). Il riepilogo somma i quantita_rimanente di tutti i lotti attivi per prodotto. Il valore del magazzino è la somma di quantita_rimanente × prezzo per ogni lotto.

Regole di integrità

- Non si può eliminare una marca se ha prodotti collegati
- Non si può eliminare un prodotto se ha giacenza > 0 o movimenti collegati
- Non si può eliminare l'ultimo utente rimasto
- La quantità di un movimento deve essere sempre > 0
- Il tipo di movimento deve essere 'carico' o 'scarico'

Script dati di esempio (src/scripts/seed.js)

Questo script popola il database con dati di test: marche, prodotti, movimenti di carico e scarico casuali. Utile per testare l'applicazione. Si esegue con: npm run dati

I dati di esempio vengono generati combinando le marche e le categorie prodotto definite in config.json.

5. Aggiornamenti Real-Time (Socket.IO)

Quando un utente modifica i dati (aggiunge una marca, crea un prodotto, registra un movimento), il server emette un evento Socket.IO che aggiorna automaticamente l'interfaccia di tutti gli altri utenti connessi, senza dover ricaricare la pagina.

Come funziona

- Il frontend si connette al server Socket.IO all'avvio (realtime.js)
- Ogni operazione CRUD chiama skipNextSocketUpdate() per evitare che chi ha fatto la modifica ricarichi i dati due volte
- Il server emette eventi specifici dopo ogni operazione (es. marca_aggiunta, prodotto_eliminato, magazzino_aggiornato)
- Il frontend ascolta questi eventi e ricarica solo i dati necessari

Caso speciale: logout automatico

Se un amministratore modifica o elimina l'account di un utente che è attualmente connesso, quell'utente viene disconnesso automaticamente con un messaggio esplicativo. Questo avviene tramite i flag username_modificato, password_modificata e utente_eliminato nelle risposte API.

Regole password

Le password degli utenti devono rispettare questi requisiti:

- Minimo 8 caratteri
- Almeno una lettera maiuscola
- Almeno una lettera minuscola
- Almeno un numero

Esempio valido: Admin123!

6. Funzionalità Speciali

Import Scarichi da PDF

Nella sezione Movimenti, il pulsante Scarica da PDF permette di caricare un PDF di un documento di lavoro officina. Il sistema legge automaticamente la sezione RICAMBI del documento ed esegue gli scarichi per ogni prodotto trovato.

- La libreria PDF.js viene caricata dal CDN solo quando serve (lazy loading)
- Il sistema cerca la sezione RICAMBI nel testo del PDF
- Estrae codice prodotto e quantità da ogni riga
- Aggrega i duplicati e invia tutto con una sola chiamata API (bulk-scarico)
- Mostra un riepilogo finale: scarichi ok, prodotti non trovati, giacenza insufficiente

Carico da Fattura PDF con AI

Il pulsante Carica da Fattura PDF usa l'intelligenza artificiale (Claude AI di Anthropic) per leggere una fattura, DDT o bolla e proporre automaticamente i movimenti di carico.

- L'utente carica il PDF nel dropzone
- Il PDF viene convertito in base64 e inviato all'API di Claude
- Claude restituisce un JSON strutturato con: numero documento, fornitore, data, lista prodotti con quantità e prezzo
- L'UI mostra la tabella delle righe estratte
- Per ogni riga, l'utente può cliccare Carica per salvare il movimento direttamente, senza aprire nessun form

Stampe

Le stampe vengono generate come HTML in un iframe invisibile e inviate direttamente alla stampante del browser.

- Stampa Riepilogo: giacenze attuali con tutti i lotti per prodotto e valore totale
- Stampa Storico: giacenze alla data selezionata
- Ogni stampa include: logo aziendale, indirizzo, P.IVA, email, telefono
- I dati aziendali vengono inseriti tramite placeholder {{company.*}} sostituiti da insertCompanyInfoPrint()

7. Installazione e Avvio

Requisiti

- Node.js versione 16 o superiore
- NPM (incluso con Node.js)
- Un server Linux (VPS) o Windows con accesso alla porta 3000

Installazione

#	Comando	Descrizione
1	cd Backend	Entra nella cartella backend
2	npm install	Installa le dipendenze (express, socket.io, bcrypt, sqlite3, cors)
3	node server.js	Avvia il server
4	(opzionale) npm run dati	Popola il DB con dati di esempio

Avvio in produzione con PM2

- npm install -g pm2
- pm2 start server.js --name magazzino
- pm2 save (salva per riavvio automatico)
- pm2 startup (configura avvio automatico al boot del server)

Dipendenze Node.js (package.json)

Pacchetto	Funzione
express	Framework web per le API REST
socket.io	Comunicazione real-time WebSocket
bcrypt	Hashing sicuro delle password
sqlite3	Driver per il database SQLite
cors	Gestione delle policy CORS per le richieste cross-origin
http (built-in)	Creazione server HTTP per Socket.IO
os (built-in)	Rilevamento IP locale della macchina

8. Dashboard

La Dashboard è la prima voce del menu e la pagina che si apre dopo il login. Serve a capire a colpo d'occhio dove intervenire: in cima c'è l'elenco "Da fare adesso", poi i numeri chiave, un grafico degli ultimi 6 mesi e gli elementi più recenti. Ogni riquadro porta direttamente alla sezione giusta.

Endpoint

Metodo	Percorso	Risposta
GET	/api/dashboard	kpi, azioni, andamento, ultimi_movimenti, top_valore, generato_il

Cose da fare (azioni)

Il backend calcola un elenco di azioni ordinate per urgenza (alta → media → info). Ogni azione contiene codice, gravità, titolo, descrizione, conteggio, fino a 5 esempi, sezione di destinazione e testo del pulsante.

Codice	Gravità	Quando compare	Porta a
esauriti	alta	Prodotti a giacenza zero che hanno già avuto movimenti	Riordino
merce_ferma	media	Lotti ancora in magazzino caricati da più di 180 giorni	Riepilogo
senza_movimenti	media	Prodotti senza alcun movimento	Movimenti
senza_marca	info	Prodotti senza marca	Prodotti
marche_vuote	info	Marche senza prodotti (eliminabili)	Marche

Numeri chiave (kpi)

marche, prodotti, movimenti, utenti, valore_magazzino, pezzi_in_giacenza, movimenti_30gg, prodotti_esauriti. L'andamento contiene, per ciascuno degli ultimi 6 mesi, i pezzi caricati e scaricati; top_valore elenca i 5 prodotti con più valore in giacenza; ultimi_movimenti gli ultimi 6 movimenti.

Dove si trova il codice

- Backend: routes/dashboardRoutes.js → controllers/dashboardController.js → services/dashboardService.js → repositories/dashboardRepository.js (sola lettura, nessuna scrittura).
- Frontend: js/features/dashboard/dashboard.js (loadDashboard, renderDashboard, dashGoTo, refreshDashboardIfActive) e css/features/dashboard.css.
- Il grafico è un SVG generato dal JavaScript: non servono librerie esterne.
- Se il real-time è attivo, refreshDashboardIfActive() aggiorna la Dashboard ad ogni evento; in ogni caso c'è il pulsante "Aggiorna" e la pagina si ricarica entrando nella sezione.
- Tutti i testi provenienti dal database vengono sottoposti a escape HTML prima di essere mostrati.

9. Eliminazione multipla

In ogni tabella eliminabile (marche, prodotti, movimenti, utenti) è possibile selezionare più righe con le checkbox e premere "Elimina selezionati". Il sistema elimina tutto ciò che è eliminabile, salta gli elementi collegati ad altri dati e alla fine mostra un riepilogo con cosa è stato eliminato e cosa no (con il motivo). Un elemento non eliminabile non blocca mai gli altri.

Endpoint

Metodo	Percorso	Note
POST	/api/marche/bulk-delete	Salta le marche con prodotti collegati

Metodo	Percorso	Note
POST	/api/prodotti/bulk-delete	Salta i prodotti con giacenza residua o movimenti collegati
POST	/api/dati/bulk-delete	Movimenti: gli scarichi vengono eliminati prima dei carichi
POST	/api/utenti/bulk-delete	Salta l'utente connesso; lascia sempre almeno un utente

Richiesta e risposta

Richiesta: { "ids": [1, 2, 3] } (per gli utenti anche "current_user" con il nome dell'utente connesso).
Risposta (sempre HTTP 200 se la richiesta è valida):

{ "success": true, "message": "1 eliminata, 1 non eliminabile.", "totale": 2, "eliminati_count": 1, "non_eliminati_count": 1, "eliminati": [{ "id": 3, "etichetta": "Vuota1" }], "non_eliminati": [{ "id": 1, "etichetta": "Honda", "motivo": "…collegato…" }] }

Se "ids" è vuoto o non valido il server risponde 400. Se nessun elemento è eliminabile la risposta è comunque 200 con eliminati_count = 0 e l'elenco dei motivi.

Come funziona nel backend

- utils/bulkDelete.js: la funzione eliminaMultipli() scorre gli id, richiama la stessa eliminazione singola di sempre (quindi con gli stessi controlli) e raccoglie eliminati / non_eliminati con il motivo. Ogni service espone eliminaMultipli(ids).
- realtime/socket.js: batch() raccoglie gli eventi Socket.IO emessi durante l'operazione e li invia una sola volta alla fine (deduplicati), evitando decine di ricaricamenti sui client. Gli eventi utente_* restano distinti per non perdere il logout mirato.
- Movimenti: gli scarichi selezionati vengono eliminati per primi perché ripristinano la quantità nei lotti; così anche i carichi selezionati diventano eliminabili. Un carico già (parzialmente) scaricato da un movimento non selezionato viene saltato.

Come funziona nel frontend

- js/shared/bulk-selection.js: BulkSelection.register({ key, tbody, endpoint, singolare, plurale, femminile, extraBody, onDone }) aggiunge toolbar, select-all (anche indeterminato), evidenziazione righe e dialoghi di conferma e riepilogo.
- Le righe devono contenere <td class="col-select"><input type="checkbox" class="bulk-check" data-id data-label>; l'intestazione contiene <input class="bulk-check-all">. Un MutationObserver riallinea le spunte dopo ogni render (filtri, ricarica, real-time).
- La selezione riguarda solo le righe visibili in tabella; dopo l'eliminazione la tabella viene ricaricata (onDone) e la selezione azzerata.
- Stili in css/shared/bulk-selection.css. Il modulo è identico nei due progetti.

Tabella	tbody	Endpoint	Ricarica dopo
Marche	marcheTableBody	POST marche/bulk-delete	loadMarche
Prodotti	prodottiTableBody	POST prodotti/bulk-delete	loadProdotti + loadMarche
Movimenti	movimentiTableBody	POST dati/bulk-delete	loadMovimenti + loadProdotti
Utenti	utentiTableBody	POST utenti/bulk-delete	loadUtenti